Java Backend Architecture

Interview Series

Chapter 14.4
CAP Theorem & Consistency Models

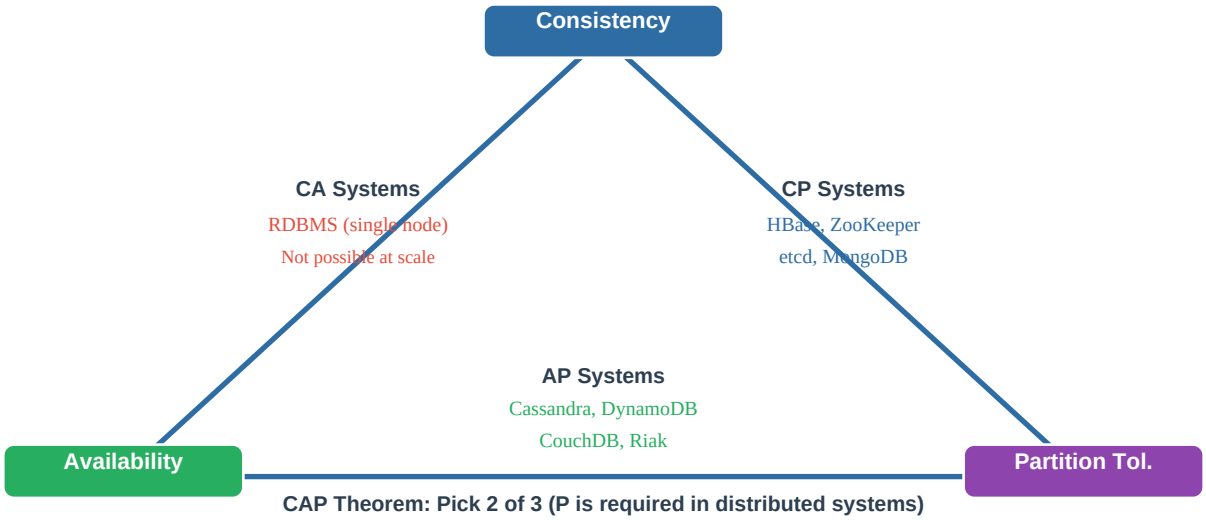
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 14.4 – CAP Theorem & Consistency Models

Overview

The CAP theorem defines the fundamental trade-offs in distributed systems. Understanding CAP, PACELC, and the spectrum of consistency models is essential for choosing the right database and designing systems that meet both functional and non-functional requirements. These concepts directly influence database selection and data architecture decisions.

CAP Theorem

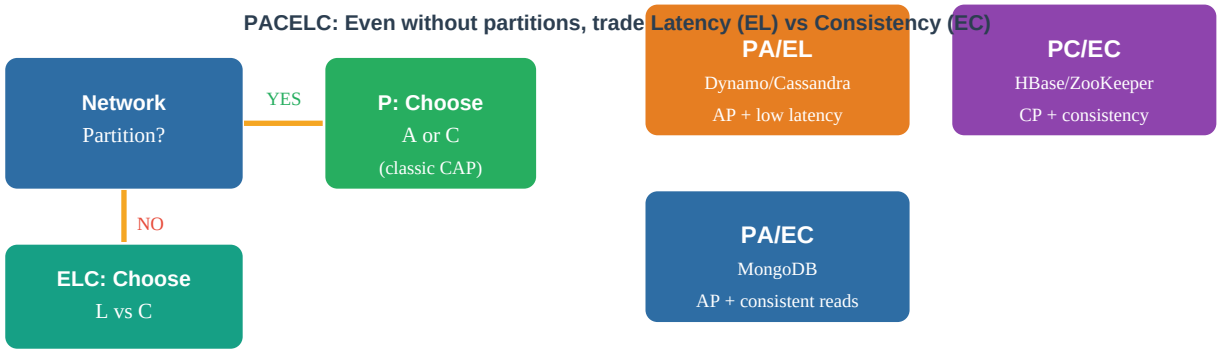


Consistency (C)	Every read receives the most recent write or an error. All nodes see the same data at the same time. Requires synchronous replication — write must propagate to all nodes before acknowledgment.
Availability (A)	Every request receives a response (not an error), but it might not be the latest data. System stays online and responsive even during node failures.
Partition Tolerance (P)	System continues operating despite network partitions (messages between nodes being dropped or delayed). In practice, partitions are unavoidable in distributed systems — you always need P.
Why CA is impossible	Since network partitions are unavoidable in distributed systems, you must have Partition Tolerance. The real choice is between Consistency and Availability during a partition: CP or AP.

CP vs AP Database Classification

Category	Behavior During Partition	Examples	Use Cases
CP Systems	Return error / refuse writes to preserve consistency	HBase, ZooKeeper, etcd, MongoDB, Spanner	Financial transactions, distributed locks, leade
AP Systems	Return potentially stale data to remain available	Cassandra, DynamoDB, CouchDB, Riak	Social feeds, product catalogs, session storag
CA Systems	Not possible at distributed scale — SPOF	Single-node RDBMS (MySQL, PostgreSQL)	ACID distributed: fits on one machine

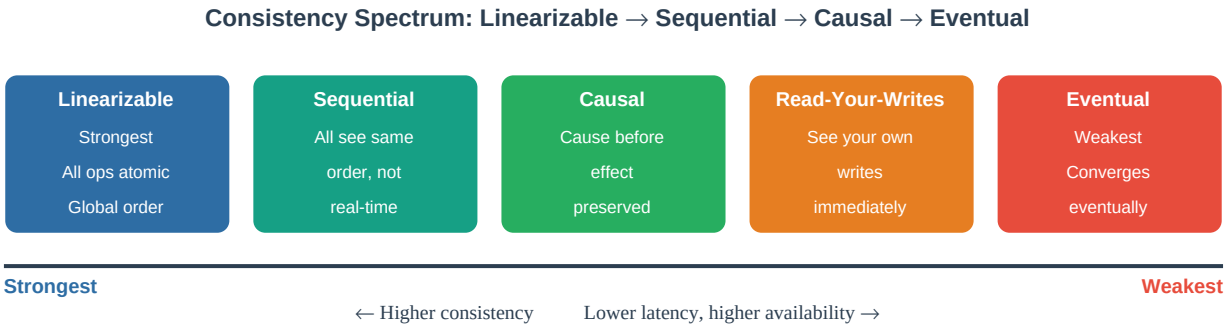
PACELC Theorem



PACELC extends CAP: even without a Partition (E=Else), a distributed system must choose between Latency (L) and Consistency (C). This captures the everyday trade-off — not just during failures.

PA/EL	Partition: Available. Normal: Low Latency. Cassandra, DynamoDB. Best performance and availability. Trades consistency for speed.
PC/EC	Partition: Consistent. Normal: Consistent. HBase, ZooKeeper, Spanner. Highest consistency guarantees. Accepts higher latency.
PA/EC	Partition: Available. Normal: Consistent. MongoDB (primary reads). AP during failure but strongly consistent during normal operation.
PC/EL	Partition: Consistent. Normal: Low Latency. Rare combination. PNUTS (Yahoo). Consistent during partitions but optimizes for latency normally.

Consistency Models Spectrum



Model	Guarantee	Latency	Examples
Linearizability	Operations appear instantaneous and globally ordered	High	etcd, ZooKeeper, Spanner
Sequential Consistency	All nodes see operations in same order, not necessarily real-time	High	Some distributed DBs
Causal Consistency	Causally related ops seen in order; concurrent ops can differ	Medium	MongoDB causal sessions
Read-Your-Writes	User always sees their own writes	Medium	DynamoDB strong reads
Monotonic Reads	If read X, will never see older version of X	Medium	Cassandra with read repair
Eventual Consistency	All replicas converge eventually; reads may be stale	Low	Cassandra ONE, S3

Dynamo-Style Tunable Consistency (W + R > N)

- N = total replicas. W = write quorum. R = read quorum
- W + R > N ensures at least one node is in both read and write quorums → overlap guarantees latest write
- Strong consistency: W = N (all must ACK write) or W + R > N
- High availability: W = 1 (fastest write), R = 1 (fastest read) — eventual consistency
- Balanced: N=3, W=2, R=2 → strong consistency, tolerates 1 node failure
- DynamoDB: ConsistencyLevel per request. Cassandra: per-query consistency level

```
// Cassandra tunable consistency example SimpleStatement stmt =
SimpleStatement.builder("SELECT * FROM users WHERE id = ?")
.setConsistencyLevel(ConsistencyLevel.QUORUM) // R quorum .build(); // Write with strong
consistency SimpleStatement write = SimpleStatement.builder("INSERT INTO users ...")
.setConsistencyLevel(ConsistencyLevel.LOCAL_QUORUM) // W within DC .build(); // DynamoDB
strongly consistent read GetItemRequest request = GetItemRequest.builder() .tableName("users")
.key(key) .consistentRead(true) // Uses 2 read units instead of 0.5 .build();
```

Write Skew and Anomalies

Write Skew	Two transactions read overlapping data, make disjoint writes, violating a constraint. Example: two doctors both read "at least 1 doctor on-call", both decide to go off-call simultaneously → 0 doctors on-call. Requires serializable isolation to prevent.
Read-Your-Writes	User makes a write, then immediately reads — they should see their own write. Violated when: subsequent read hits a replica that hasn't received the write yet. Fix: route reads from same user to same replica, or use strong consistency for that user.
Monotonic Reads	If a user reads a value, subsequent reads should not return an older value. Violated when: requests hit different replicas at different lag states. Fix: pin user to same replica for duration of session.
Phantom Read	Transaction T1 reads a set of rows matching a condition. T2 inserts/deletes rows matching that condition. T1 re-reads and sees different set. Prevented by: SERIALIZABLE isolation, predicate locks, or MVCC with range locking.

Interview Q&A; — CAP Theorem & Consistency Models

Q1. What does the CAP theorem state?

The CAP theorem states that a distributed system can guarantee at most 2 of 3 properties: Consistency (every read sees the latest write), Availability (every request gets a response), and Partition Tolerance (system works despite network partitions). Since network partitions are unavoidable in distributed systems, the practical choice is between Consistency and Availability during a partition: CP or AP.

Q2. Why is CA (Consistency + Availability without Partition Tolerance) impossible at scale?

Partition Tolerance cannot be sacrificed in a distributed system — network failures, dropped packets, and node failures are facts of life. A CA system would need to stop accepting requests whenever a partition occurs (to maintain consistency) or accept requests without knowing if another partition member has processed conflicting writes (losing consistency). At scale with multiple datacenters, partitions are guaranteed to occur.

Q3. Name 3 CP databases and explain why they sacrifice availability.

HBase: during region server failure, regions go offline until master reassigns — no reads/writes to those regions. ZooKeeper: requires quorum (majority of nodes) for reads/writes — minority partition cannot serve requests. etcd: Raft consensus requires quorum; minority partition returns errors. All sacrifice availability during partitions to ensure every successful operation returns consistent data.

Q4. Name 3 AP databases and explain the consistency trade-off.

Cassandra: with ConsistencyLevel.ONE, reads/writes always succeed even with node failures — but may read stale data from a lagging replica. DynamoDB: eventually consistent reads (default) may return data up to 1 second stale. CouchDB: multi-master replication, accepts writes on any node, reconciles conflicts later using revision history. All prioritize uptime over immediate consistency.

Q5. What is the PACELC theorem and how does it extend CAP?

PACELC: if there is a Partition, choose between Availability and Consistency (PA or PC). Else (no partition), choose between Latency and Consistency (EL or EC). This captures the everyday trade-off: even without failures, synchronous replication for consistency adds latency. Cassandra is PA/EL (available during partition, low latency normally). Spanner is PC/EC (consistent during partition, consistent reads normally).

Q6. What is linearizability?

Linearizability (atomic consistency) is the strongest consistency model. All operations appear to execute instantaneously at some point between their start and end times. All clients see operations in the same real-time order. Analogy: like a single-threaded system — operations are globally ordered. Examples: etcd and ZooKeeper guarantee linearizable reads/writes. Price: requires synchronous quorum consensus (Raft, Paxos) — higher latency.

Q7. What is eventual consistency and when is it acceptable?

Eventual consistency: given no new updates, all replicas will converge to the same value eventually (typically ms to seconds). Reads may return stale data during convergence. Acceptable for: social media likes/view counts (approximate is fine), product recommendations, DNS (TTL-based staleness), email delivery status. NOT acceptable for: bank balance, inventory counts, order processing, concurrent counter updates needing exact values.

Q8. What is the quorum formula W + R > N?

N = replication factor. W = write quorum (must-acknowledge nodes). R = read quorum. W + R > N ensures that the read set and write set overlap by at least 1 node — that node has the latest write. Example: N=3, W=2, R=2: overlap = W+R-N = 1. Strong consistency with N=3, W=1 requires R=3 (every node must be read).

Performance vs consistency knob.

Q9. What is Cassandra's consistency level system?

Per-query consistency: ONE (1 node ACK), QUORUM (majority), LOCAL_QUORUM (majority in local DC), ALL (all replicas), EACH_QUORUM (quorum in each DC). Write consistency controls how many replicas must ACK before success. Read consistency controls how many replicas are queried (highest-version wins). Combine W + R > N for strong consistency. Spring Data Cassandra: set via @ConsistencyLevel annotation or CqlTemplate.

Q10. What is write skew and how does it violate consistency?

Write skew: two concurrent transactions read overlapping data, make decisions, and write to non-overlapping rows — but together they violate an invariant. Classic example: constraint "at least 1 doctor on-call", T1 reads 2 doctors on-call and sets Doctor A off-call, T2 reads 2 doctors on-call and sets Doctor B off-call simultaneously — result: 0 doctors. Requires SERIALIZABLE isolation level to prevent. Snapshot isolation does NOT prevent it.

Q11. What is the difference between read-your-writes and monotonic reads?

Read-your-writes: after a write, the same user always sees that write on subsequent reads. Violated when replica lag causes user to read from a replica that hasn't received the write. Fix: route reads for same user to same replica for some TTL. Monotonic reads: once you read value V, you never read an older value. Violated by round-robin reads across replicas with different lag. Fix: pin user to same replica for session duration.

Q12. How does DynamoDB handle consistency?

DynamoDB replicates to 3 AZs. Default reads: eventually consistent (may be 1 second stale). Strongly consistent reads: set ConsistentRead=true — reads from all 3 replicas and returns latest version. Cost: strong reads = 2× read capacity units. Writes: always synchronously written to 2+ replicas before ACK. DynamoDB transactions (2023): provide ACID across items/tables using 2-phase commit — behaves as CP for transactional workloads.

Q13. What is causal consistency and how is it stronger than eventual?

Causal consistency preserves cause-and-effect ordering: if operation A causally depends on operation B (B happened before A), then all clients see B before A. Concurrent operations (no causal relationship) can be seen in any order. Example: user updates their profile, then posts — followers always see the updated profile before the post. Stronger than eventual (which allows any ordering), weaker than sequential (which requires global total order). MongoDB Causal Consistency sessions implement this.

Q14. How does ZooKeeper guarantee linearizable reads?

ZooKeeper uses Zab (ZooKeeper Atomic Broadcast) consensus protocol. All writes go through the elected leader. Reads: by default served from any node (may be stale). sync() + read: forces read from leader, guaranteeing linearizability. Watches: linearizable event notifications. Used for: distributed locks, leader election, config management. Quorum: majority of N nodes must be available (N=3: 2 needed, N=5: 3 needed).

Q15. What is vector clocks and how do they track causality?

Vector clock: each node maintains a counter for every other node. On event: increment own counter. On send: attach entire vector. On receive: merge by taking max per component. Causality: VC(A) < VC(B) means A happened before B. Concurrent: neither VC(A) < VC(B) nor VC(B) < VC(A). Amazon Dynamo used vector clocks for conflict detection. DynamoDB replaced with simpler last-write-wins. Riak still uses vector clocks.

Q16. What is the difference between CP and CA consistency in distributed systems?

CP: Consistent + Partition Tolerant. During network partition, system refuses requests rather than returning potentially inconsistent data. Availability sacrificed. CA: Consistent + Available. Only possible without network partitions — i.e., single-node systems. Not a valid distributed system category at scale. SQL databases (MySQL, PostgreSQL) are CA only as single-node deployments; become CP when run as multi-master clusters.

Q17. How does Spanner achieve external consistency (linearizability) globally?

Google Spanner uses TrueTime API with GPS + atomic clocks to bound clock uncertainty. Assigns commit timestamps within bounded uncertainty. Waits out the uncertainty interval (typically 1-7ms) before committing — ensures no two commits get the same timestamp. Paxos consensus across multiple datacenters. Result: globally consistent, serializable distributed transactions. Cost: added latency for uncertainty wait + consensus.

Q18. What is conflict-free replicated data type (CRDT)?

CRDT: data structure designed for eventual consistency where concurrent updates can be merged automatically without conflicts. G-Counter (grow-only): each node has its own counter, total = sum of all. PN-Counter: positive + negative G-counters, supports decrement. LWW-Register: last-write-wins with timestamp. G-Set: only add, no remove. OR-Set: supports remove. Used in: Redis, Riak, collaborative editors (Google Docs uses CRDT-like OT).

Q19. What is the difference between strong consistency and strict serializability?

Strong consistency = linearizability: every operation appears instantaneous, real-time ordered. Serializability: transactions execute as if in some serial order — not necessarily real-time. Strict serializability = serializability + real-time ordering (the strongest guarantee). Spanner provides strict serializability. PostgreSQL SERIALIZABLE provides serializability (SSI - Serializable Snapshot Isolation). Most databases offer something weaker: REPEATABLE READ or READ COMMITTED.

Q20. What is monotonic write consistency?

Monotonic write: writes from a single client are applied in the order they were issued. If client sends Write(x=1) then Write(x=2), no replica ever applies them in reverse order. Violation: client sends writes to different replicas with different lag — replica B might receive and apply Write(x=2) before Write(x=1). Fix: route all writes from same session through same node (primary), use timestamps for ordering.

Q21. How does eventual consistency work in practice for S3?

S3 offered read-after-write consistency for new objects (PUT then GET → returns object). For overwrites and deletes, S3 provided eventual consistency until 2020 when AWS upgraded to strong read-after-write consistency for all operations. Implementation: S3 uses a distributed metadata store with synchronous replication across AZs. Objects are stored redundantly — reads go to any replica. Strong consistency added by making reads wait for all replicas to confirm the latest version.

Q22. What is the BASE model and how does it contrast with ACID?

BASE: Basically Available (AP databases — always responds), Soft State (state may change over time even without new input due to eventual consistency), Eventually Consistent (replicas converge over time). Contrast: ACID — Atomicity (all-or-nothing), Consistency (DB invariants maintained), Isolation (transactions don't interfere), Durability (committed writes persist). ACID: CP systems (RDBMS). BASE: AP systems (Cassandra, DynamoDB). Modern systems often mix both.

Q23. What is the Paxos protocol and how does it ensure consistency?

Paxos: consensus protocol ensuring all nodes agree on a single value despite failures. Phases: Prepare (proposer sends ballot number, acceptors promise not to accept older proposals), Accept (proposer sends value, acceptors accept if ballot still highest), Learn (learners receive accepted value). Requires quorum

(majority) in each phase. Guarantees no two nodes decide different values (safety). Liveness: progress if majority available. Used in: Chubby, Zookeeper (Zab variant), etcd (Raft variant).

Q24. What is Raft consensus and how does it differ from Paxos?

Raft is a consensus protocol designed for understandability. Leader election: candidates request votes from majority; elected leader has highest log term+index. Log replication: leader sends AppendEntries to followers; committed when majority ACK. Log safety: leader only commits entries from current term. Compared to Paxos: Raft is easier to implement and reason about (explicit leader, simpler log structure). Used in: etcd, CockroachDB, TiKV, RabbitMQ (quorum queues).

Q25. How does MongoDB handle consistency?

MongoDB default: reads from primary (strong consistency). Secondary reads: may be stale (configurable readPreference). Write concern: w:1 (primary ACK), w:majority (majority ACK), w:0 (fire and forget). Read concern: "local" (latest write on queried node), "majority" (majority-committed data — strongly consistent), "linearizable" (strongest, slowest). Causal consistency sessions: track operationTime and clusterTime for read-your-writes.

Q26. What is a split-brain scenario and how does it relate to CAP?

Split-brain: network partition divides cluster into two halves, each believing it is the primary (both accept writes). Result: conflicting updates, data divergence. CAP connection: CP systems avoid split-brain by requiring quorum — minority partition refuses writes. AP systems allow split-brain but must reconcile conflicts on partition heal (last-write-wins, vector clocks, CRDTs). Prevention: fencing tokens, STONITH (Shoot The Other Node In The Head), quorum-based leader election.

Q27. What is the difference between optimistic and pessimistic consistency controls?

Pessimistic: lock data before accessing, prevent concurrent modification. High contention → lock wait queues → lower throughput. Safe for high-conflict workloads. Optimistic: allow concurrent reads/writes, detect conflicts at commit time (via version numbers or timestamps). Retry on conflict. Higher throughput for low-conflict workloads. CAS (Compare-And-Swap) is optimistic. SELECT FOR UPDATE is pessimistic. DynamoDB condition expressions = optimistic.

Q28. How do you choose between CP and AP for your application?

Choose CP when: financial transactions (bank transfers must be exact), inventory management (overselling is unacceptable), distributed locks (leader election requires exactly-one guarantee), user authentication (stale password could allow old password reuse). Choose AP when: social feeds (slightly stale is fine), product search (stale inventory counts acceptable), session data (can use eventual, user unlikely to notice), analytics (approximate counts fine).

Q29. What is Multi-Version Concurrency Control (MVCC) and how does it relate to consistency?

MVCC maintains multiple versions of data simultaneously. Readers see consistent snapshot (their transaction start timestamp) without blocking writers. Writers create new versions without blocking readers. PostgreSQL, MySQL InnoDB, Oracle, CockroachDB use MVCC. Enables: snapshot isolation (consistent reads across multi-row queries), read-committed without read locks. MVCC provides good read consistency without full serializable guarantees.

Q30. How does CRDTs enable collaborative editing (like Google Docs)?

Collaborative editor using CRDT: each user's edits are represented as operations that can be merged in any order without conflicts. Sequence CRDT (like RGA or LSEQ): each character has unique identifier, operations are commutative and associative. User A inserts "hello" at position 0. User B simultaneously inserts "world" at position 0. CRDT merge produces deterministic result without server coordination. Google Docs uses Operational Transformation (OT), similar in spirit.

Q31. What is a phantom read and which isolation level prevents it?

Phantom read: transaction T1 queries "SELECT * FROM orders WHERE status='pending'", gets N rows. Transaction T2 inserts a new pending order. T1 re-queries, sees N+1 rows — the "phantom" row. This violates REPEATABLE READ semantics. Prevention: SERIALIZABLE isolation (predicate locks or SSI range detection), or MVCC with range locking (PostgreSQL). MySQL InnoDB: gap locks in REPEATABLE READ level also prevent phantom reads.

Q32. How does Apache Cassandra implement tunable consistency?

Cassandra: each write/read can specify consistency level independently. ALL: all replicas. QUORUM: majority of all DCs. LOCAL_QUORUM: majority in local DC only (avoids cross-DC latency). ONE/TWO/THREE: specific number of replicas. ANY: at least 1 (includes hints). Hinted handoff: if target node down, write stored as hint on coordinator, delivered when node recovers. Read repair: on quorum read, if replicas disagree, background repair to latest value. Anti-entropy: Merkle tree comparison for full repair.

Q33. What is eventual consistency in DNS?

DNS is a classic eventually consistent system. When you update a DNS record (e.g., change IP address), the change propagates through the hierarchy: authoritative NS → recursive resolvers → client caches. TTL determines max staleness (300s typical → change takes up to 5 minutes to propagate globally). During propagation: some clients get old IP, some get new. This is acceptable for DNS because the alternative (synchronous global propagation) would make DNS too slow.

Q34. How does Google Bigtable handle consistency?

Bigtable: single master per cluster + multiple tablet servers. Strong consistency within a tablet (single tablet server owns row range). Reads/writes for a given row go to exactly one tablet server — no concurrent writes to same row across servers. No cross-row transactions (Spanner added this). Failure: master detects dead tablet server via Chubby locks, reassigns tablets → brief unavailability (CP model). HBase is open-source Bigtable clone with same model.

Q35. What is the role of consensus in distributed consistency?

Consensus (Paxos, Raft) allows a group of nodes to agree on a single value despite failures. Essential for: leader election (single leader for write serialization), distributed transactions (all nodes commit or abort), membership changes (agree on cluster configuration). Cost: requires quorum round-trip for every decision → high latency. CP databases use consensus for correctness. AP databases avoid consensus for performance — accept divergence.

Q36. How does Apache Kafka provide consistency guarantees?

Kafka: messages within a partition are totally ordered (sequential consistency within partition). Leader replica owns writes; followers replicate asynchronously. ISR (In-Sync Replicas): subset of replicas within configurable lag. acks=all: producer waits for all ISR to ACK. min.insync.replicas=2: requires at least 2 ISR before ACK. Exactly-once: idempotent producer (dedup by sequence number) + transactions (atomic multi-partition writes). Consumer reads only committed offsets.

Q37. What is the difference between serializability and snapshot isolation?

Snapshot isolation: each transaction reads from a consistent snapshot at transaction start. Prevents: dirty reads, non-repeatable reads, phantom reads for read-only transactions. Does NOT prevent: write skew (concurrent writes to different rows violating invariant). Serializability: equivalent to some serial execution of transactions. Prevents all anomalies including write skew. PostgreSQL SERIALIZABLE uses SSI (Serializable Snapshot Isolation) to add write skew detection on top of MVCC without excessive locking.